

!pip install diffusers transformers accelerate peft

Collecting diffusers
 Downloading diffusers-0.36.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.3)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.12.0)
Collecting peft
 Downloading peft-0.18.0-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: importlib_metadata in /usr/lib/python3/dist-packages (from diffusers) (4.6.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from diffusers) (3.20.0)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from diffusers) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from diffusers) (0.36.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from diffusers) (2.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from diffusers) (2025.11.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from diffusers) (2.32.4)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.12/dist-packages (from diffusers) (0.7.0)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from diffusers) (12.0.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.9.0+cpu)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers) (4.12.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers) (2025.11.12)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers) (3.11)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0->diffusers) (0.16.
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.34.0->diffuser
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.34.0
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.34.0->diff
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.1.6)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->diffusers) (3.4.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->diffusers) (2.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->acceler
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0->accelerate) (3.0.
Downloading diffusers-0.36.0-py3-none-any.whl (4.6 MB)
 4.6/4.6 MB 74.0 MB/s eta 0:00:00
Downloading peft-0.18.0-py3-none-any.whl (556 kB)
 556.4/556.4 kB 40.1 MB/s eta 0:00:00
Installing collected packages: diffusers, peft
Successfully installed diffusers-0.36.0 peft-0.18.0

```
import os
import json
import torch
import numpy as np
from pathlib import Path
from PIL import Image, ImageDraw, ImageFont
from typing import List, Dict, Tuple
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from diffusers import StableDiffusionPipeline, DDPMSScheduler
from transformers import CLIPTextModel, CLIPTokenizer
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')
```

/usr/local/lib/python3.12/dist-packages/jax/_src/cloud_tpu_init.py:86: UserWarning: Transparent hugepages are not enabled. TPU runtime warnings.warn(
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your version
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your version

```
class TextOverlayHelper:
    """Helper class to add text overlays to generated images"""

    def __init__(self):
        self.load_fonts()

    def load_fonts(self):
        """Load system fonts with fallbacks"""
        self.huge_font = None
        self.large_font = None
        self.medium_font = None

        font_paths = [
            "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
            "/System/Library/Fonts/Arial.ttf",
            "C:\\Windows\\Fonts\\arial.ttf",
            "/usr/share/fonts/truetype/liberation/LiberationSans-Bold.ttf",
        ]

        for font_path in font_paths:
            if Path(font_path).exists():
                try:
                    self.huge_font = ImageFont.truetype(font_path, 48)
                    self.large_font = ImageFont.truetype(font_path, 36)
                    self.medium_font = ImageFont.truetype(font_path, 28)
                    return
                except:
                    continue

        # Fallback
```

```

        self.huge_font = ImageFont.load_default()
        self.large_font = ImageFont.load_default()
        self.medium_font = ImageFont.load_default()

def draw_text_with_shadow(self, draw, text, position, font, fill='black', shadow_color='white', shadow_offset=3):
    """Draw text with shadow for readability"""
    x, y = position
    # Draw shadow
    draw.text((x + shadow_offset, y + shadow_offset), text, font=font, fill=shadow_color)
    # Draw main text
    draw.text((x, y), text, font=font, fill=fill)

def add_flowchart_labels(self, img: Image.Image) -> Image.Image:
    """Add text labels to flowchart image"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    height = img.height
    box_height = height // 5

    labels = ["START", "INPUT", "PROCESS", "OUTPUT", "END"]
    positions = [
        (img.width // 2, box_height // 2),
        (img.width // 2, box_height + box_height // 2),
        (img.width // 2, 2 * box_height + box_height // 2),
        (img.width // 2, 3 * box_height + box_height // 2),
        (img.width // 2, 4 * box_height + box_height // 2),
    ]

    for label, (x, y) in zip(labels, positions):
        bbox = draw.textbbox((0, 0), label, font=self.large_font)
        text_width = bbox[2] - bbox[0]
        text_height = bbox[3] - bbox[1]
        self.draw_text_with_shadow(draw, label,
                                   (x - text_width // 2, y - text_height // 2),
                                   self.large_font, fill='#003366', shadow_color='white', shadow_offset=2)

    return img

def add_bar_chart_labels(self, img: Image.Image, categories: List[str], values: List[int]) -> Image.Image:
    """Add text labels to bar chart"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    width = img.width
    height = img.height

    # Title
    title = "CHART"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (width // 2 - text_width // 2, 20),
                               self.huge_font, fill='#003366', shadow_color='white')

    # Category labels at bottom
    bar_width = width // (len(categories) + 1)
    for i, cat in enumerate(categories):
        x_pos = bar_width * (i + 1) + bar_width // 2
        y_pos = height - 60
        bbox = draw.textbbox((0, 0), cat, font=self.large_font)
        text_width = bbox[2] - bbox[0]
        self.draw_text_with_shadow(draw, cat, (x_pos - text_width // 2, y_pos),
                                   self.large_font, fill='black', shadow_color='white')

    # Value labels on bars
    for i, (cat, val) in enumerate(zip(categories, values)):
        x_pos = bar_width * (i + 1) + bar_width // 2
        y_pos = height - 150 - (val * 2)
        val_text = str(val)
        bbox = draw.textbbox((0, 0), val_text, font=self.large_font)
        text_width = bbox[2] - bbox[0]
        self.draw_text_with_shadow(draw, val_text, (x_pos - text_width // 2, y_pos),
                                   self.large_font, fill='darkred', shadow_color='white')

    return img

def add_line_graph_labels(self, img: Image.Image, line_labels: List[str], months: List[str]) -> Image.Image:
    """Add text labels to line graph"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    # Title
    title = "TREND"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (img.width // 2 - text_width // 2, 20),
                               self.huge_font, fill='#003366', shadow_color='white')

    # Month labels
    month_spacing = img.width // (len(months) + 2)
    for i, month in enumerate(months):
        x_pos = month_spacing * (i + 1) + 40
        y_pos = img.height - 50
        bbox = draw.textbbox((0, 0), month, font=self.medium_font)
        text_width = bbox[2] - bbox[0]
        self.draw_text_with_shadow(draw, month, (x_pos - text_width // 2, y_pos),
                                   self.medium_font, fill='black', shadow_color='white')

```

```

        # Line legend
        legend_x = 30
        legend_y = 80
        for i, label in enumerate(line_labels):
            y_offset = i * 50
            self.draw_text_with_shadow(draw, label, (legend_x, legend_y + y_offset),
                                      self.medium_font, fill='black', shadow_color='white')

    return img

def add_pie_chart_labels(self, img: Image.Image, labels: List[str], sizes: List[int]) -> Image.Image:
    """Add text labels to pie chart"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    # Title
    title = "DISTRIBUTION"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (img.width // 2 - text_width // 2, 20),
                              self.huge_font, fill='#003366', shadow_color='white')

    # Legend on the right
    legend_x = img.width - 250
    legend_y = 100

    for i, (label, size) in enumerate(zip(labels, sizes)):
        y_offset = i * 60
        text = f"{label}: {size}%"
        self.draw_text_with_shadow(draw, text, (legend_x, legend_y + y_offset),
                                    self.medium_font, fill='black', shadow_color='white')

    return img

def add_scatter_labels(self, img: Image.Image, x_label: str, y_label: str) -> Image.Image:
    """Add axis labels to scatter plot"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    # Title
    title = "CORRELATION"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (img.width // 2 - text_width // 2, 20),
                              self.huge_font, fill='#003366', shadow_color='white')

    # X label
    bbox = draw.textbbox((0, 0), x_label, font=self.large_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, x_label, (img.width // 2 - text_width // 2, img.height - 60),
                              self.large_font, fill='black', shadow_color='white')

    # Y label (rotated effect - draw at side)
    self.draw_text_with_shadow(draw, y_label, (20, img.height // 2 - 50),
                              self.large_font, fill='black', shadow_color='white')

    return img

def add_network_labels(self, img: Image.Image, node_names: List[str]) -> Image.Image:
    """Add labels to network nodes"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    # Title
    title = "NETWORK"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]
    self.draw_text_with_shadow(draw, title, (img.width // 2 - text_width // 2, 20),
                              self.huge_font, fill='#003366', shadow_color='white')

    # Node positions
    positions = [
        (img.width // 2, img.height // 2), # Center
        (img.width // 2, 100),           # Top
        (img.width - 80, img.height // 2), # Right
        (img.width // 2, img.height - 80), # Bottom
        (80, img.height // 2),             # Left
    ]

    for name, (x, y) in zip(node_names, positions):
        bbox = draw.textbbox((0, 0), name, font=self.large_font)
        text_width = bbox[2] - bbox[0]
        text_height = bbox[3] - bbox[1]
        self.draw_text_with_shadow(draw, name, (x - text_width // 2, y - text_height // 2),
                                    self.large_font, fill='white', shadow_color='black', shadow_offset=2)

    return img

def add_venn_labels(self, img: Image.Image, set_a: str, set_b: str, intersection: str) -> Image.Image:
    """Add labels to Venn diagram"""
    img = img.copy()
    draw = ImageDraw.Draw(img)

    # Title
    title = "SET THEORY"
    bbox = draw.textbbox((0, 0), title, font=self.huge_font)
    text_width = bbox[2] - bbox[0]

```

```
text_width = bbox[2] - bbox[0]
self.draw_text_with_shadow(draw, title, (img.width // 2 - text_width // 2, 20),
                           self.huge_font, fill='#003366', shadow_color='white')

# Left circle label
bbox = draw.textbbox((0, 0), set_a, font=self.huge_font)
text_width = bbox[2] - bbox[0]
self.draw_text_with_shadow(draw, set_a, (img.width // 3 - text_width // 2, img.height // 2 - 40),
                           self.huge_font, fill='#FF6B6B', shadow_color='white', shadow_offset=2)

# Right circle label
bbox = draw.textbbox((0, 0), set_b, font=self.huge_font)
text_width = bbox[2] - bbox[0]
self.draw_text_with_shadow(draw, set_b, (2 * img.width // 3 - text_width // 2, img.height // 2 - 40),
                           self.huge_font, fill='#4ECDC4', shadow_color='white', shadow_offset=2)

# Intersection label
bbox = draw.textbbox((0, 0), intersection, font=self.large_font)
text_width = bbox[2] - bbox[0]
self.draw_text_with_shadow(draw, intersection, (img.width // 2 - text_width // 2, img.height // 2 + 20),
                           self.large_font, fill='black', shadow_color='white', shadow_offset=2)

return img
```

```
class AcademicVisualizationGenerator:
    """Generate clean visualizations for training, add text after generation"""

    def __init__(self, output_dir: str = "./academic_viz_dataset"):
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(exist_ok=True)
        self.images_dir = self.output_dir / "images"
        self.images_dir.mkdir(exist_ok=True)
        self.metadata = []
        self.text_helper = TextOverlayHelper()

    def create_flowchart(self) -> tuple:
        """Generate clean flowchart - text added after"""
        fig, ax = plt.subplots(figsize=(8, 10), dpi=100)

        from matplotlib.patches import FancyBboxPatch

        boxes = [
            (0.35, 0.85, 0.3, 0.08),
            (0.35, 0.68, 0.3, 0.08),
            (0.35, 0.51, 0.3, 0.08),
            (0.35, 0.34, 0.3, 0.08),
            (0.35, 0.17, 0.3, 0.08),
        ]

        colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A', '#95E1D3']

        for i, (x, y, w, h) in enumerate(boxes):
            box = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.01",
                                edgecolor='black', facecolor=colors[i], linewidth=3,
                                transform=ax.transAxes)
            ax.add_patch(box)

        arrow_props = dict(arrowstyle='->', lw=3, color='black')
        for i in range(len(boxes) - 1):
            y_from = boxes[i][1]
            y_to = boxes[i+1][1] + boxes[i+1][3]
            ax.annotate('', xy=(0.5, y_to), xytext=(0.5, y_from),
                        arrowprops=arrow_props, xycoords='axes fraction',
                        textcoords='axes fraction')

        ax.set_xlim(0, 1)
        ax.set_ylim(0, 1)
        ax.axis('off')

        import io
        buf = io.BytesIO()
        fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
        buf.seek(0)
        img = Image.open(buf).convert('RGB')
        img.load()
        buf.close()
        plt.close(fig)

        # ADD TEXT OVERLAY
        img = self.text_helper.add_flowchart_labels(img)

        description = "Flowchart with five colored boxes labeled START, INPUT, PROCESS, OUTPUT, END with arrows connecting them vertically"

        return img, description

    def create_bar_chart(self) -> tuple:
        """Generate bar chart - text added after"""
        fig, ax = plt.subplots(figsize=(10, 6), dpi=100)

        x_pos = [0, 1, 2, 3]
        values = [65, 78, 92, 87]
        colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A']
        categories = ['Q1', 'Q2', 'Q3', 'Q4']

        bars = ax.bar(x_pos, values, color=colors, edgecolor='black', linewidth=2.5, width=0.6)

        ax.set_xticks([])
```

```

ax.set_ylim(0, 110)
ax.grid(axis='y', alpha=0.3, linewidth=1.5)

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.set_yticks([])

import io
buf = io.BytesIO()
fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
buf.seek(0)
img = Image.open(buf).convert('RGB')
img.load()
buf.close()
plt.close(fig)

# ADD TEXT OVERLAY
img = self.text_helper.add_bar_chart_labels(img, categories, values)

description = f"Bar chart with four bars labeled {' '.join(categories)} with values {' '.join(map(str, values))}."

return img, description

def create_line_graph(self) -> tuple:
    """Generate line graph - text added after"""
    fig, ax = plt.subplots(figsize=(10, 6), dpi=100)

    months = ['J', 'F', 'M', 'A', 'M', 'J']
    line1 = [20, 30, 45, 60, 75, 90]
    line2 = [80, 75, 65, 55, 50, 45]

    ax.plot(range(len(months)), line1, marker='o', linewidth=3, markersize=10,
            color='#FF6B6B')
    ax.plot(range(len(months)), line2, marker='s', linewidth=3, markersize=10,
            color='#4ECDC4')

    ax.grid(True, alpha=0.3, linewidth=1.5)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_visible(False)
    ax.spines['bottom'].set_visible(False)

    import io
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
    buf.seek(0)
    img = Image.open(buf).convert('RGB')
    img.load()
    buf.close()
    plt.close(fig)

    # ADD TEXT OVERLAY
    month_names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
    img = self.text_helper.add_line_graph_labels(img, ['Line 1', 'Line 2'], month_names)

    description = "Line graph with two trend lines over six months showing upward and downward trends."

    return img, description

def create_pie_chart(self) -> tuple:
    """Generate pie chart - text added after"""
    fig, ax = plt.subplots(figsize=(8, 8), dpi=100)

    sizes = [30, 25, 25, 20]
    colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A']
    explode = (0.1, 0, 0, 0)

    ax.pie(sizes, colors=colors, explode=explode, startangle=90)
    ax.axis('equal')

    import io
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
    buf.seek(0)
    img = Image.open(buf).convert('RGB')
    img.load()
    buf.close()
    plt.close(fig)

    # ADD TEXT OVERLAY
    labels = ['Comp A', 'Comp B', 'Comp C', 'Comp D']
    img = self.text_helper.add_pie_chart_labels(img, labels, sizes)

    description = f"Pie chart divided into four sections: {' '.join([f'{l} {s}%' for l, s in zip(labels, sizes)])}."

    return img, description

def create_scatter_plot(self) -> tuple:
    """Generate scatter plot - text added after"""
    fig, ax = plt.subplots(figsize=(8, 6), dpi=100)

    np.random.seed(42)
    x = np.random.randn(100) * 2 + 5
    y = x * 1.5 + np.random.randn(100) * 2

```



```

ax.scatter(x, y, s=100, alpha=0.7, color='#45B7D1', edgecolors='black', linewidth=1)

z = np.polyfit(x, y, 1)
p = np.poly1d(z)
ax.plot(x, p(x), "r--", linewidth=3)

ax.grid(True, alpha=0.3, linewidth=1.5)
ax.set_xticks([])
ax.set_yticks([])

import io
buf = io.BytesIO()
fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
buf.seek(0)
img = Image.open(buf).convert('RGB')
img.load()
buf.close()
plt.close(fig)

# ADD TEXT OVERLAY
img = self.text_helper.add_scatter_labels(img, 'Variable X', 'Variable Y')

description = "Scatter plot showing correlation between Variable X and Variable Y with 100 blue points and red trend line."

return img, description

def create_network_diagram(self) -> tuple:
    """Generate network - text added after"""
    fig, ax = plt.subplots(figsize=(8, 8), dpi=100)

    from matplotlib.patches import Circle

    positions = {
        'center': (0.5, 0.5),
        'top': (0.5, 0.85),
        'right': (0.8, 0.5),
        'bottom': (0.5, 0.15),
        'left': (0.2, 0.5)
    }

    colors = ['#45B7D1', '#FF6B6B', '#4ECDC4', '#FFA07A', '#95E1D3']

    for key in ['top', 'right', 'bottom', 'left']:
        x_vals = [positions['center'][0], positions[key][0]]
        y_vals = [positions['center'][1], positions[key][1]]
        ax.plot(x_vals, y_vals, 'gray', linewidth=3, zorder=1)

    for i, (key, (x, y)) in enumerate(positions.items()):
        radius = 0.08 if key == 'center' else 0.06
        circle = Circle((x, y), radius, color=colors[i], ec='black', linewidth=2, zorder=2)
        ax.add_patch(circle)

    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.set_aspect('equal')
    ax.axis('off')

    import io
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
    buf.seek(0)
    img = Image.open(buf).convert('RGB')
    img.load()
    buf.close()
    plt.close(fig)

    # ADD TEXT OVERLAY
    node_names = ['HUB', 'Node A', 'Node B', 'Node C', 'Node D']
    img = self.text_helper.add_network_labels(img, node_names)

    description = "Network diagram with central HUB node connected to Node A, Node B, Node C, and Node D in hub-and-spoke topology"

    return img, description

def create_venn_diagram(self) -> tuple:
    """Generate Venn - text added after"""
    fig, ax = plt.subplots(figsize=(8, 6), dpi=100)

    circle1 = plt.Circle((0.35, 0.5), 0.25, color='#FF6B6B', alpha=0.4)
    circle2 = plt.Circle((0.65, 0.5), 0.25, color='#4ECDC4', alpha=0.4)

    ax.add_patch(circle1)
    ax.add_patch(circle2)

    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.set_aspect('equal')
    ax.axis('off')

    import io
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=100, facecolor='white')
    buf.seek(0)
    img = Image.open(buf).convert('RGB')
    img.load()
    buf.close()

```

```

plt.close(fig)

# ADD TEXT OVERLAY
img = self.text_helper.add_venn_labels(img, 'Set A', 'Set B', 'AnB')

description = "Venn diagram with two overlapping circles representing Set A and Set B with intersection labeled AnB."

return img, description

def generate_dataset(self, samples_per_type: int = 5) -> None:
    """Generate complete dataset"""
    generators = [
        self.create_flowchart,
        self.create_bar_chart,
        self.create_line_graph,
        self.create_pie_chart,
        self.create_scatter_plot,
        self.create_network_diagram,
        self.create_venn_diagram
    ]

    for gen_func in generators:
        for i in range(samples_per_type):
            img, description = gen_func()

            filename = f"{gen_func.__name__[7:]}_{i}.png"
            filepath = self.images_dir / filename
            img.save(filepath)

            self.metadata.append({
                "image": filename,
                "text": description,
                "type": gen_func.__name__[7:]
            })
            print(f"Generated: {filename}")

    metadata_path = self.output_dir / "metadata.json"
    with open(metadata_path, 'w') as f:
        json.dump(self.metadata, f, indent=2)
    print(f"\nDataset saved to {self.output_dir}")
    print(f"Total samples: {len(self.metadata)}")

```

```

class AcademicVisualizationDataset(Dataset):
    """PyTorch Dataset for image-text pairs"""

    def __init__(self, metadata_path: str, images_dir: str, image_size: int = 512):
        with open(metadata_path, 'r') as f:
            self.metadata = json.load(f)

        self.images_dir = Path(images_dir)
        self.image_size = image_size

        self.transform = transforms.Compose([
            transforms.Resize((image_size, image_size)),
            transforms.ToTensor(),
            transforms.Normalize([0.5], [0.5])
        ])

    def __len__(self):
        return len(self.metadata)

    def __getitem__(self, idx):
        item = self.metadata[idx]
        image_path = self.images_dir / item["image"]

        image = Image.open(image_path).convert('RGB')
        image = self.transform(image)

        text = item["text"]

        return {"image": image, "text": text}

```

```

class AcademicVizModelTrainer:
    """Fine-tune Stable Diffusion for academic visualizations"""

    def __init__(self, model_id: str = "runwayml/stable-diffusion-v1-5", device: str = None):
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")
        self.model_id = model_id

        print(f"Loading model from {model_id}...")
        self.tokenizer = CLIPTokenizer.from_pretrained(model_id, subfolder="tokenizer")
        self.text_encoder = CLIPTextModel.from_pretrained(model_id, subfolder="text_encoder").to(self.device)

        self.vae = None
        self.unet = None
        self.scheduler = None
        self.pipeline = None
        self.text_helper = TextOverlayHelper()

    def setup_training(self, learning_rate: float = 5e-5, enable_lora: bool = True):
        """Initialize training components"""
        from diffusers import AutoencoderKL, UNet2DConditionModel

        print("Loading VAE and UNet...")
        self.vae = AutoencoderKL.from_pretrained(self.model_id, subfolder="vae").to(self.device)

```

```

self.unet = UNet2DConditionModel.from_pretrained(self.model_id, subfolder="unet").to(self.device)

self.vae.requires_grad_(False)
self.text_encoder.requires_grad_(False)

if enable_lora:
    print("Applying LoRA adaptation...")
    try:
        from peft import LoraConfig, inject_adapter_in_model

        lora_config = LoraConfig(
            r=16,
            lora_alpha=32,
            target_modules=["to_k", "to_q", "to_v", "to_out.0"],
            lora_dropout=0.1,
            bias="none"
        )

        inject_adapter_in_model(lora_config, self.unet)

        trainable_params = sum(p.numel() for p in self.unet.parameters() if p.requires_grad)
        total_params = sum(p.numel() for p in self.unet.parameters())
        print(f"LoRA applied: {trainable_params:,} / {total_params:,} trainable")

    except Exception as e:
        print(f"LoRA not available: {e}")

self.scheduler = DDPMSScheduler.from_pretrained(self.model_id, subfolder="scheduler")
self.optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, self.unet.parameters()),
    lr=learning_rate
)

def train(self, dataloader: DataLoader, epochs: int = 5, save_dir: str = "./checkpoints"):
    """Train the model"""
    save_path = Path(save_dir)
    save_path.mkdir(exist_ok=True)

    self.unet.train()
    self.text_encoder.eval()

    for epoch in range(epochs):
        total_loss = 0
        progress_bar = tqdm(dataloader, desc=f"Epoch {epoch+1}/{epochs}")

        for batch in progress_bar:
            images = batch["image"].to(self.device)
            texts = batch["text"]

            with torch.no_grad():
                tokens = self.tokenizer(
                    texts,
                    padding="max_length",
                    max_length=self.tokenizer.model_max_length,
                    truncation=True,
                    return_tensors="pt"
                ).input_ids.to(self.device)

                text_embeddings = self.text_encoder(tokens)[0]

            with torch.no_grad():
                latents = self.vae.encode(images).latent_dist.sample() * 0.18215

            noise = torch.randn_like(latents)
            bsz = latents.shape[0]
            timesteps = torch.randint(0, 1000, (bsz,), device=self.device).long()

            noisy_latents = self.scheduler.add_noise(latents, noise, timesteps)

            noise_pred = self.unet(
                noisy_latents,
                timesteps,
                encoder_hidden_states=text_embeddings
            ).sample

            loss = torch.nn.functional.mse_loss(noise_pred, noise)
            loss.backward()

            self.optimizer.step()
            self.optimizer.zero_grad()

            total_loss += loss.item()
            progress_bar.set_postfix({"loss": total_loss / (progress_bar.n + 1)})

        checkpoint_path = save_path / f"checkpoint_epoch_{epoch+1}"
        self.unet.save_pretrained(checkpoint_path)
        print(f"Checkpoint saved: {checkpoint_path}")

def generate_and_annotate(self, prompt: str, annotation_type: str = "flowchart",
                          num_inference_steps: int = 50, guidance_scale: float = 7.5) -> Image.Image:
    """Generate image and add text overlay based on type"""
    if self.pipeline is None:
        from diffusers import StableDiffusionPipeline

        self.pipeline = StableDiffusionPipeline.from_pretrained(
            self.model_id,
            unet=self.unet,

```



```

        tokenizer=self.tokenizer,
        text_encoder=self.text_encoder,
        vae=self.vae,
        scheduler=self.scheduler
    ).to(self.device)

    with torch.no_grad():
        image = self.pipeline(
            prompt,
            num_inference_steps=num_inference_steps,
            guidance_scale=guidance_scale
        ).images[0]

    # Add text overlay based on type
    if annotation_type == "flowchart":
        image = self.text_helper.add_flowchart_labels(image)
    elif annotation_type == "bar":
        image = self.text_helper.add_bar_chart_labels(image, ['Q1', 'Q2', 'Q3', 'Q4'], [65, 78, 92, 87])
    elif annotation_type == "line":
        image = self.text_helper.add_line_graph_labels(image, ['Line 1', 'Line 2'],
                                                            ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'])

    elif annotation_type == "scatter":
        image = self.text_helper.add_scatter_labels(image, 'Variable X', 'Variable Y')
    elif annotation_type == "network":
        image = self.text_helper.add_network_labels(image, ['HUB', 'Node A', 'Node B', 'Node C', 'Node D'])
    elif annotation_type == "pie":
        image = self.text_helper.add_pie_chart_labels(image, ['Comp A', 'Comp B', 'Comp C', 'Comp D'],
                                                            [30, 25, 25, 20])

    elif annotation_type == "venn":
        image = self.text_helper.add_venn_labels(image, 'Set A', 'Set B', 'AnB')

    return image

```

```

if __name__ == "__main__":
    print("=" * 70)
    print("Hybrid Approach: Stable Diffusion + PIL Text Overlay")
    print("=" * 70)

    print("\n[STEP 1] Generating synthetic dataset with text overlays...")
    generator = AcademicVisualizationGenerator(output_dir="./academic_viz_dataset")
    generator.generate_dataset(samples_per_type=5)

    print("\n[STEP 2] Setting up data loader...")
    dataset = AcademicVisualizationDataset(
        metadata_path="./academic_viz_dataset/metadata.json",
        images_dir="./academic_viz_dataset/images",
        image_size=512
    )
    dataloader = DataLoader(dataset, batch_size=4, shuffle=True, num_workers=0)
    print(f"DataLoader ready with {len(dataset)} samples")

    print("\n[STEP 3] Initializing trainer...")
    trainer = AcademicVizModelTrainer()
    trainer.setup_training(learning_rate=5e-5, enable_lora=True)

    print("\n[STEP 4] Starting training...")
    trainer.train(dataloader, epochs=5, save_dir="./checkpoints")

    print("\n[STEP 5] Testing inference with text overlay...")
    test_cases = [
        ("flowchart showing vertical process flow", "flowchart"),
        ("bar chart with four colored bars", "bar"),
        ("scatter plot with blue points", "scatter"),
    ]

    output_dir = Path("./generated_images")
    output_dir.mkdir(exist_ok=True)

    for prompt, viz_type in test_cases:
        print(f"\nGenerating {viz_type}: {prompt}")
        image = trainer.generate_and_annotate(prompt, annotation_type=viz_type, num_inference_steps=30)
        image.save(output_dir / f"generated_{viz_type}.png")
        print(f"Saved: {output_dir / f'generated_{viz_type}.png'}")

    print("\n" + "=" * 70)
    print("Training complete! Generated images with readable text labels.")
    print("=" * 70)

```


=====

Hybrid Approach: Stable Diffusion + PIL Text Overlay

=====

[STEP 1] Generating synthetic dataset with text overlays...

Generated: flowchart_0.png

Generated: flowchart_1.png

Generated: flowchart_2.png

Generated: flowchart_3.png

Generated: flowchart_4.png

Generated: bar_chart_0.png

Generated: bar_chart_1.png